

MACHINE LEARNING AVEC PYTHON

Support de cours — Introduction pratique aux concepts,
aux algorithmes et à l'écosystème scikit-learn

Niveau : Débutant / Intermédiaire

Prérequis : bases de Python, notions de mathématiques (algèbre, statistiques)

8 pages — Document pédagogique

1. Introduction au Machine Learning

Le **Machine Learning** (apprentissage automatique) est une branche de l'intelligence artificielle qui permet à un système d'apprendre à partir de données, sans être explicitement programmé pour chaque tâche. Plutôt que d'écrire des règles fixes, on entraîne un modèle mathématique à reconnaître des motifs (patterns) dans les données pour ensuite faire des prédictions sur de nouvelles données.

Définition clé

Un algorithme de Machine Learning "apprend" une fonction f qui associe une entrée X à une sortie Y à partir d'exemples (X, Y) , afin de généraliser cette fonction à des données jamais vues.

Pourquoi Python ?

Python s'est imposé comme le langage de référence du Machine Learning grâce à sa syntaxe simple, sa communauté active et un écosystème de bibliothèques matures :

- **Simplicité** — syntaxe claire, idéale pour prototyper rapidement
- **Écosystème riche** — NumPy, Pandas, Scikit-learn, TensorFlow, PyTorch
- **Communauté** — documentation abondante, forums, tutoriels
- **Interopérabilité** — s'intègre facilement avec le Big Data et le cloud

Domaines d'application

- Reconnaissance d'images et de la parole
- Systèmes de recommandation (Netflix, Amazon)
- Détection de fraude bancaire
- Diagnostic médical assisté
- Véhicules autonomes, chatbots, finance quantitative

2. Les grands types d'apprentissage

2.1 — Apprentissage supervisé

Le modèle apprend à partir de données **étiquetées** (chaque exemple possède une réponse connue). On distingue deux sous-catégories :

- **Régression** — prédire une valeur continue (ex. : prix d'une maison)
- **Classification** — prédire une catégorie (ex. : email spam ou non)

2.2 — Apprentissage non supervisé

Le modèle explore des données **non étiquetées** pour en découvrir la structure sous-jacente, sans réponse prédéfinie.

- **Clustering** — regrouper des données similaires (segmentation clients)
- **Réduction de dimension** — simplifier les données (ACP / PCA)
- **Détection d'anomalies** — identifier des comportements atypiques

2.3 — Apprentissage par renforcement

Un **agent** apprend par essai-erreur en interagissant avec un environnement, recevant des récompenses ou pénalités selon ses actions. Utilisé pour les jeux, la robotique ou l'optimisation de systèmes.

Type	Données	Exemple
Supervisé	Étiquetées	Prédire un prix
Non supervisé	Non étiquetées	Segmenter des clients
Renforcement	Récompenses	Jeu vidéo, robotique

3. L'écosystème Python pour le ML

Un projet de Machine Learning en Python repose généralement sur un ensemble de bibliothèques complémentaires, chacune dédiée à une étape du pipeline.

NumPy — Calcul numérique

Fournit les tableaux multidimensionnels (arrays) et les opérations mathématiques vectorisées, à la base de toutes les autres bibliothèques.

Pandas — Manipulation de données

Structure de données tabulaires (DataFrame) pour charger, nettoyer, filtrer et explorer des jeux de données.

Matplotlib / Seaborn — Visualisation

Permettent de créer des graphiques pour explorer les données et interpréter les résultats des modèles.

Scikit-learn — Machine Learning classique

Bibliothèque de référence offrant une interface unifiée pour la régression, la classification, le clustering, la réduction de dimension et l'évaluation des modèles.

TensorFlow / PyTorch — Deep Learning

Bibliothèques dédiées aux réseaux de neurones profonds, utilisées pour la vision par ordinateur, le traitement du langage naturel, etc.

Installation de l'environnement

```
pip install numpy pandas matplotlib seaborn scikit-learn

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
```

4. Le pipeline d'un projet Machine Learning

Tout projet de Machine Learning suit généralement une méthodologie en plusieurs étapes, itérative et non strictement linéaire :

- **1. Collecte des données** — rassembler les données brutes (fichiers, bases, API)
- **2. Nettoyage / préparation** — valeurs manquantes, doublons, encodage des variables
- **3. Analyse exploratoire (EDA)** — statistiques descriptives, visualisations
- **4. Division du jeu de données** — ensemble d'entraînement / ensemble de test
- **5. Choix et entraînement du modèle** — sélection d'un algorithme adapté au problème
- **6. Évaluation** — mesure de la performance avec des métriques adaptées
- **7. Optimisation** — réglage des hyperparamètres (GridSearch, etc.)
- **8. Déploiement** — mise en production du modèle (API, application)

Division train / test

Pour évaluer honnêtement un modèle, on sépare les données en un ensemble d'**entraînement** (généralement 70 à 80 %) et un ensemble de **test** (20 à 30 %), afin de vérifier sa capacité à généraliser sur des données inédites.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(X_train.shape, X_test.shape)
```

Bonne pratique

Ne jamais utiliser les données de test pour entraîner ou ajuster le modèle : cela fausserait l'évaluation et donnerait une image trop optimiste de la performance réelle.

5. Algorithmes supervisés courants

Régression linéaire

Modélise une relation linéaire entre variables explicatives et une variable cible continue. Simple et interprétable, c'est souvent le point de départ d'une analyse.

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

Régression logistique et classification

Malgré son nom, la régression logistique est un algorithme de **classification** qui estime la probabilité d'appartenance à une classe.

```
from sklearn.linear_model import LogisticRegression

clf = LogisticRegression()
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
```

Autres algorithmes fréquents

- **Arbres de décision** — règles de décision hiérarchiques, faciles à interpréter
- **Forêts aléatoires (Random Forest)** — ensemble d'arbres, robuste et performant
- **K plus proches voisins (KNN)** — classe selon les exemples les plus proches
- **SVM** — sépare les classes par une frontière optimale
- **Gradient Boosting (XGBoost)** — très performant sur données tabulaires

6. Algorithmes non supervisés

K-Means — Clustering

Regroupe les observations en **k** groupes homogènes en minimisant la distance entre chaque point et le centre de son groupe (centroïde).

```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X)
clusters = kmeans.labels_
```

Clustering hiérarchique

Construit une hiérarchie de groupes représentée sous forme d'arbre (dendrogramme), sans avoir à fixer le nombre de clusters à l'avance.

Analyse en Composantes Principales (ACP / PCA)

Technique de réduction de dimension qui projette les données sur de nouveaux axes (composantes) tout en conservant le maximum de variance. Utile pour la visualisation et pour réduire le bruit avant l'entraînement.

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X_reduced = pca.fit_transform(X)
```

À retenir

Le choix entre apprentissage supervisé et non supervisé dépend avant tout de la disponibilité (ou non) d'étiquettes fiables dans les données.

7. Évaluer et améliorer un modèle

Métriques de classification

- **Exactitude (accuracy)** — proportion de prédictions correctes
- **Précision** — parmi les prédictions positives, combien sont correctes
- **Rappel (recall)** — parmi les cas réellement positifs, combien sont détectés
- **F1-score** — moyenne harmonique entre précision et rappel

Métriques de régression

- **MAE** — erreur absolue moyenne
- **MSE / RMSE** — erreur quadratique (pénalise davantage les grosses erreurs)
- **R²** — proportion de variance expliquée par le modèle

```
from sklearn.metrics import accuracy_score, classification_report

print(accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Surapprentissage et sous-apprentissage

Le **surapprentissage** (overfitting) survient quand un modèle apprend trop précisément les données d'entraînement, y compris leur bruit, et généralise mal. Le **sous-apprentissage** (underfitting) survient quand le modèle est trop simple pour capturer les tendances des données.

Validation croisée

La **validation croisée** (cross-validation) divise les données en plusieurs sous-ensembles (folds) pour entraîner et évaluer le modèle plusieurs fois, offrant une estimation plus fiable de sa performance.

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(model, X, y, cv=5)
print(scores.mean())
```


8. Exemple complet et conclusion

Pipeline complet en quelques lignes

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# 1. Chargement des données
data = pd.read_csv("donnees.csv")
X = data.drop("cible", axis=1)
y = data["cible"]

# 2. Division train / test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Entraînement du modèle
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# 4. Évaluation
y_pred = model.predict(X_test)
print("Précision :", accuracy_score(y_test, y_pred))
```

Points clés à retenir

- Le ML apprend des motifs à partir de données plutôt que des règles fixes
- Trois grandes familles : supervisé, non supervisé, renforcement
- Scikit-learn offre une interface unifiée : fit(), predict(), score()
- Toujours séparer entraînement et test pour évaluer honnêtement un modèle
- La validation croisée et le bon choix de métriques évitent les erreurs d'interprétation

Pour aller plus loin

- Documentation officielle de Scikit-learn (scikit-learn.org)
- Feature engineering et sélection de variables
- Réseaux de neurones avec TensorFlow / Keras ou PyTorch
- Déploiement de modèles via une API (FastAPI, Flask)

Conclusion

Le Machine Learning avec Python repose sur un équilibre entre compréhension théorique des algorithmes et maîtrise pratique des outils. La meilleure façon de progresser reste la pratique régulière sur des jeux de données variés.